Examen Restanta / Marire POO

Întrebări

Ce se întâmplă la execuția următorului program ?

```
#include <iostream>
int global_data;
class Test {
    int x;
public:
    Test() { x++; }
    void* operator new(size_t size) { return &global_data; }
    void operator delete(void*) { }

};
int main() {
    Test * t;
    for (int tr = 0; tr < 100; tr++)
    {
        t = new Test();
        delete t;
    }
    t = new Test();
    std::cout << *(int*)t << std::endl;
    return 1;
}
```

Choose

Ce se întâmplă la execuția următorului program?

```
#include <iostream>
class A {
public:
    ~A() { printf("~A()"); }
};
void b() {
    throw 15;
}
int main() {
    try {
        A a;
        b();
    }
    catch (int& x) {
        printf("%d", x);
    }
    return 0;
}
```

Choose

Ce se întâmplă la execuția următorului program?

```
#include <unordered_map>
class MyDB {
    static int x;
    std::unordered_map<const char*, int> m;
public:
    MyDB(const char* s) { x++; m[s] = x; };
    auto begin() { return m.begin(); }
};
int MyDB::x = 0;
int main() {
    for (auto s : { "a", "b" }) {
        printf("%s ", MyDB(s).begin()->first);
        printf("%d ", MyDB(s).begin()->second);
    }
    return 0;
}
```

Choose

Ce se întâmplă la execuția următorului program?

```
#include <iostream>
using namespace std;

int main()
{
    int x = 10;
    auto f = [&](int value) { x += value; };
    f(10);
    std::cout << x;
    return 1;
}
```

Choose

Ce se întâmplă la execuția următorului program?

```
#include <iostream>
using namespace std;

class Test
{
    int x;
public:
    Test(int _x): x(_x) { }
    int GetX() { return x; }
};
int main()
{
    Test t;
    std::cout << t.GetX();
    return 1;
}
```

Choose

Ce se întâmplă când se dau comenzile de compilare și execuție pentru următorul program (cu C++17)?

```
#include <iostream>
class C {
public:
    C() noexcept(false) { throw 2; }
    ~C() noexcept(false) { throw 5; }
};

int main()
{
    if (noexcept(C())) {
        std::cout << "alpha; ";
    }
    if (!noexcept(std::declval<C>().~C())) {
        std::cout << "beta; ";
    }
    std::cout << "gama.";
    return 0;
}
```

Choose

Ce se va întâmpla la rularea codului de mai jos?

```
#include <iostream>

unsigned long long int operator"" _transform(unsigned long long int x) {
    return x * 2;
}

int main() {
    std::cout << a_transform << " ";
    std::cout << 24_transform << " ";
    std::cout << 1.5_transform << " ";
    std::cout << -2_transform * 3;
    return 0;
}
```

Choose

Ce se va întâmpla la execuția codului de mai jos?

```
#include <iostream>
class A {
protected:
    int key;
    A() { std::cout << "constructor A; "; }
};
class B : A {
protected:
    int value;
    B() { std::cout << "constructor B; "; }
};
class C : protected B {
public:
    C() { std::cout << "constructor C; "; }
    void setImplicitValues() { this->key = 0; this->value = 3; }
    void printImplicitValues() { std::cout << this->key << " " << this->value; }
};
int main() {
    C c;
    c.setImplicitValues();
    c.printImplicitValues();
    return 0;
}
```

Choose

Ce se va întâmpla la executarea codului de mai jos?

```
1   #include <iostream>
2   template <typename T>
3   struct Base {
4       void f() {
5           std::cout << "base ";
6       }
7       void g() {
8           static_cast<T*>(this)->f();
9       }
10  };
11  struct Derived : Base<Derived> {
12      void f() {
13          std::cout << "derived ";
14      }
15  };
16  int main() {
17      Derived obj;
18      obj.f();
19      obj.g();
20      return 0;
21  }
```

Choose

Polimorfismul la timpul rulării poate fi obținut prin:

Choose

Ce se întâmplă la execuția următorului program pe un sistem x86 (32 bit)?

```
#include <iostream>
using namespace std;

class A
{
    static int x;
    int y;
    int * ptr;
};
int main() {
    A a;
    cout << sizeof(a) << " " << sizeof(A*) << endl;
    return 0;
}
```

Choose

Ce se întâmplă la execuția următorului program ?

```
#include <iostream>
using namespace std;

class Data { };
int main()
{
    std::cout << sizeof(Data);
    return 1;
}
```

Choose

Ce se întâmplă la execuția următorului program ?

```
#include <iostream>
using namespace std;

class MyTest: public Mytest
{
public:
    MyTest() { std::cout << "MyTest,"; }
};
int main()
{
    MyTest t;
    return 0;
}
```

Choose

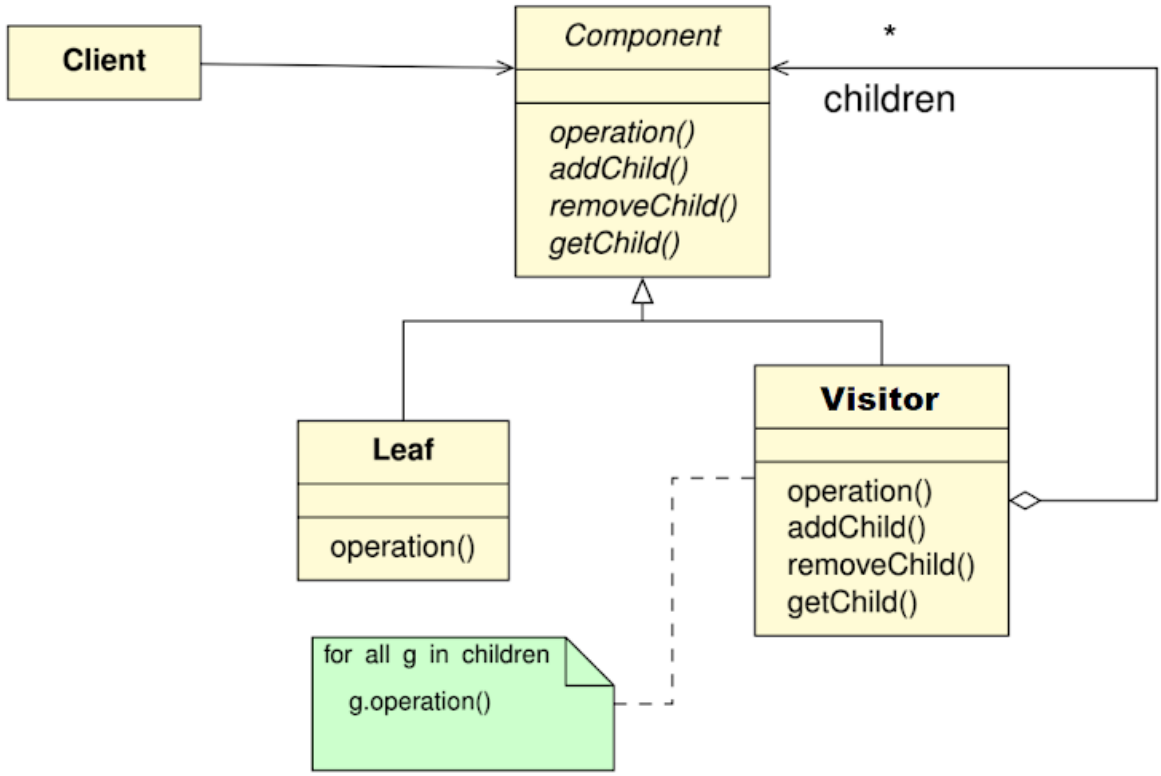
Ce se întâmplă la execuția următorului program ?

```
#include <iostream>
using namespace std;

class Numar {
    int x;
public:
    Numar(int value): x(value) { }
    void friend operator+=(Numar n1, Numar n2) { n1.x += n2.x; };
    int GetX() { return x; }
};
int main()
{
    Numar n1(5), n2(6);
    n1 += n2;
    n2 += n1;
    n1 += n2;
    std::cout << n1.GetX();
    return 0;
}
```

Choose

Ce reprezinta următorul șablon ?



Choose

A copy of your responses will be emailed to the address you provided.

Back Submit

Never submit passwords through Google Forms.

reCAPTCHA
Privacy Terms

This content is neither created nor endorsed by Google. Report Abuse - Terms of Service - Privacy Policy

Google Forms